IoT · AI · SMART HOME

Full Smart Home with AI

An End-to-End IoT Monitoring and AI Analytics Platform

Developed by
AHMED ALMADHOUN

AI Engineering Student
Islamic University of Gaza

ESP32 • Wokwi • MQTT • ThingSpeak • Blynk IoT • Jupyter Notebook • CSV database

Version 2.0
July 2026

OVERVIEW

Table of Contents

Project summary

The project consists of three independent yet complementary IoT pipelines, all fed by the same ESP32 sensor network. Part One streams sensor readings through the MQTT protocol into a Jupyter Notebook environment, where the data is stored locally, processed, and prepared for AI-based analysis and future prediction models. Part Two sends real-time sensor data directly to ThingSpeak for cloud-based visualization and live charting. Part Three connects the ESP32 to the Blynk IoT app, giving the user a mobile interface to both view live sensor data and manually control the system's actuators.

01 · OVERVIEW

Project Overview

AI Full Smart Home is a simulated IoT project built on the Wokwi platform around an ESP32 microcontroller. The system reads data from six sensors — temperature, humidity, gas, motion, light, and distance — and transmits the collected sensor data through three independent pipelines, each demonstrating a common IoT communication architecture used in real-world smart systems.

- **Figure 1** The figure illustrates the complete architecture of the proposed smart home system: the first part shows the data collection path based on the MQTT protocol, feeding into a Jupyter Notebook environment for local storage and intelligent processing; the second part depicts the cloud-based visualization path using the ThingSpeak IoT platform; and the third part presents user control via the Blynk application.

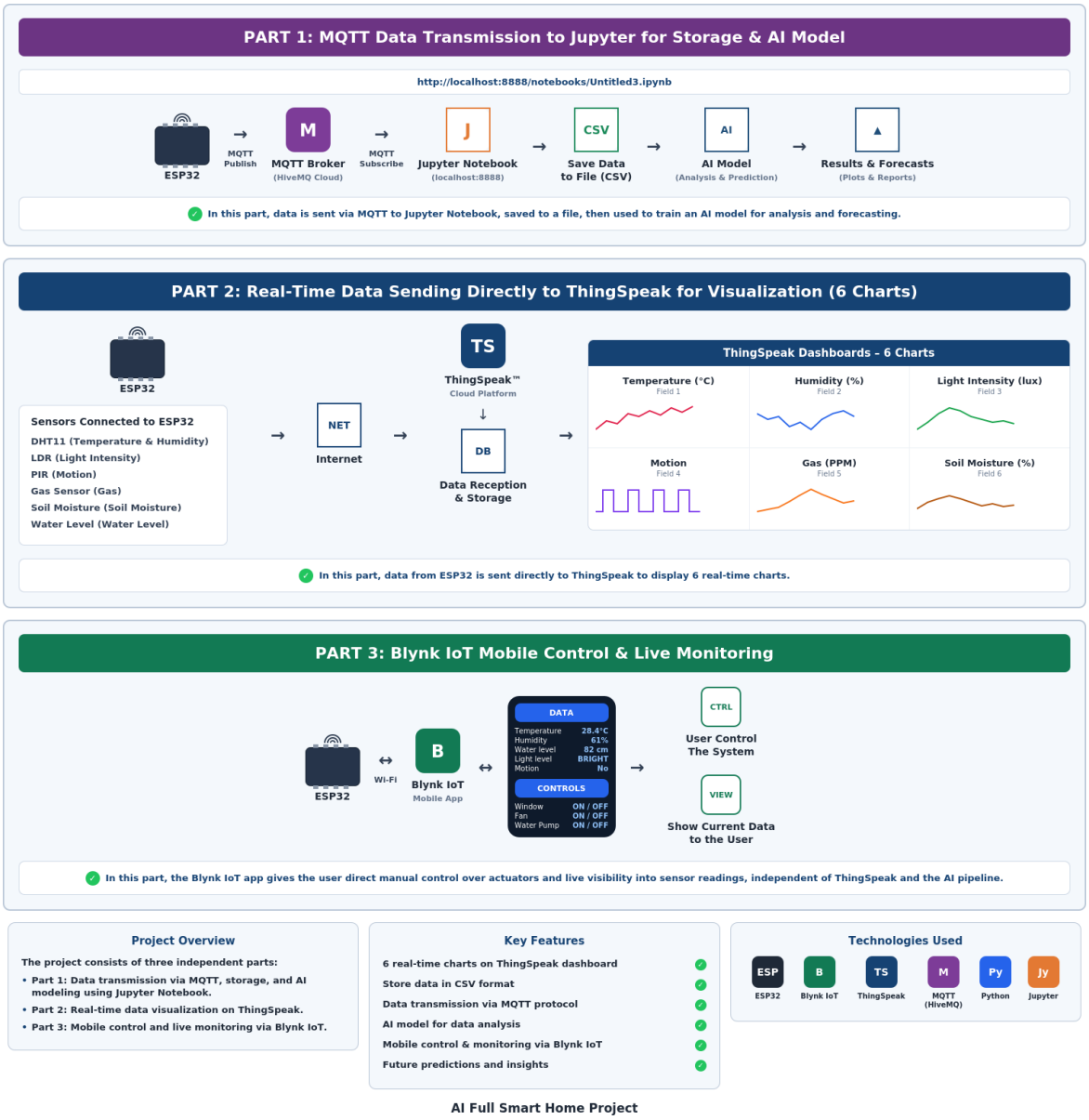

Figure 1 — Complete system architecture: the three independent pipelines branching from the ESP32 sensor network

Data Flow — Part One: ESP32 Sensors → MQTT Broker → Jupyter Notebook → CSV Storage → AI Model

Data Flow — Part Two: ESP32 Sensors → ThingSpeak Cloud → Live IoT Dashboard

Data Flow — Part Three: ESP32 Sensors → Blynk IoT → User Control & Live Data Display

Why three separate pipelines?

- **Part One** focuses on data acquisition by collecting sensor readings through the MQTT protocol, storing them locally as structured datasets, and preparing them for AI and machine learning applications.
- **Part Two** demonstrates reliable real-time monitoring by transmitting sensor data directly to the cloud for instant visualization through ThingSpeak.
- **Part Three** gives the user a mobile control panel through the Blynk IoT app, combining live sensor readings with manual control of the window, fan, lighting, and water pump.
- Separating the three pipelines improves modularity, simplifies debugging, and allows each subsystem to evolve independently without affecting the others.

02 · HARDWARE

Hardware Architecture & Sensor Network

The entire circuit is built and simulated inside Wokwi, a browser-based simulator for ESP32 / Arduino projects. It allows the full sensor network, wiring, and firmware logic to be tested without any physical hardware, while still connecting to real internet services such as ThingSpeak, Blynk IoT, and MQTT brokers.

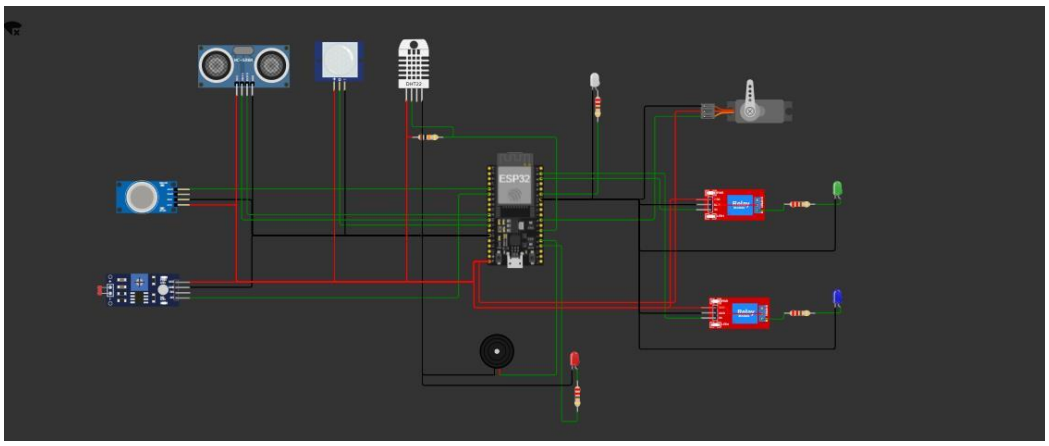

Figure 2 — Wokwi circuit diagram: ESP32 wired to all sensors and actuators

Component	Role in the system
ESP32 Dev Module	Central microcontroller — Wi-Fi, sensor reading, MQTT/HTTP publishing
DHT22 Sensor	Measures ambient temperature and humidity
Ultrasonic Sensor (HC-SR04)	Measures distance for proximity / level detection
MQ-2 Gas Sensor	Detects gas concentration for safety monitoring
PIR Motion Sensor	Detects movement within the monitored area
LDR (Light Sensor)	Measures ambient light intensity
Dual Relay Module	Switch the water pump and fan actuators on/off
Servo Motor	Controls the window
Buzzer + Alarm LED	Local alert for gas, fire, or intrusion conditions

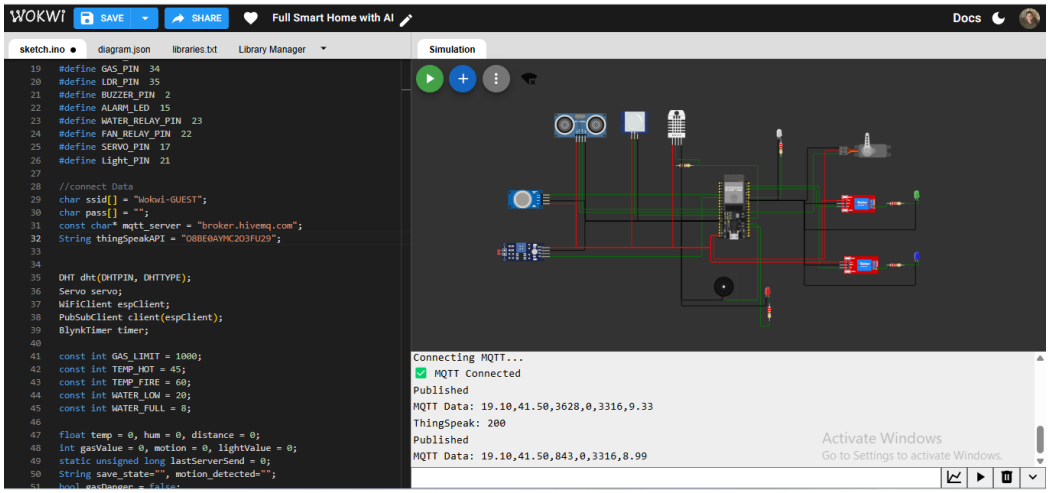

Figure 3 — Wokwi simulation running: firmware connecting to MQTT and publishing

> **Note — Safety Logic**
>
> Critical operating thresholds (gas concentration, temperature, and water level) are implemented directly in the ESP32 firmware. This enables immediate local responses through the buzzer, alarm LED, servo motor, and relay-controlled actuators before cloud services or AI analysis are involved.

03 · PART ONE

Part One — MQTT to Jupyter & AI Pipeline

ESP32 → MQTT Broker → Jupyter Notebook → CSV → AI Model

This first, independent part of the project focuses on collecting reliable data for artificial intelligence applications. An ESP32 controller transmits sensor data via the MQTT protocol to a Python environment running within a Jupyter Notebook, where the data is processed, stored, and prepared for machine learning tasks.

Pipeline stages

- **Publish —** the ESP32 unit sends sensor readings to a specific topic on a public MQTT broker.
- **Subscribe —** a Python script running inside Jupyter Notebook subscribes to the same topic using the paho-mqtt library.
- **Parse —** each incoming MQTT message is decoded and split into its six numeric values.
- **Store —** the parsed values are appended as a new row to a local sensor_data.csv file using pandas.
- **Analyze —** the accumulated CSV dataset becomes the input for an AI/ML model that can detect patterns and generate predictions.

Running this pipeline required launching a local Jupyter server and installing the necessary Python packages (paho-mqtt, pandas, and matplotlib).

Jupyter Notebook Implementation

A local Jupyter server was launched to host the notebooks used for subscribing to the MQTT protocol and processing data.

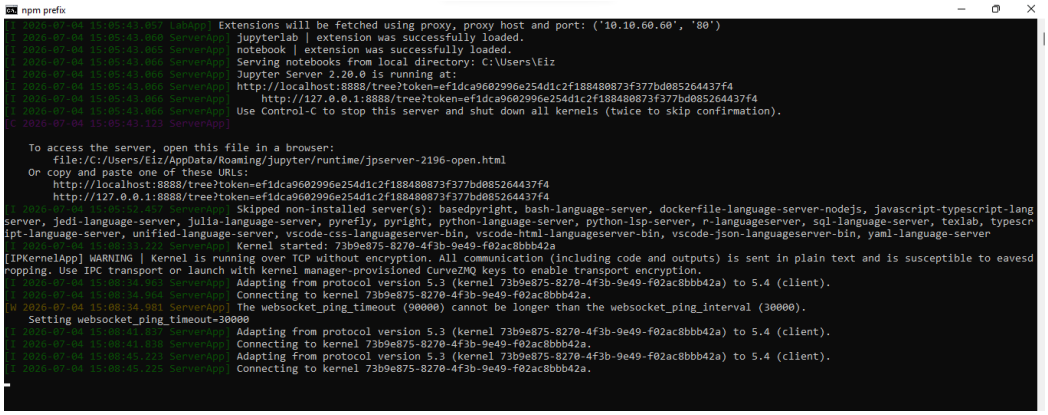

Figure 4 — Jupyter server startup: local access URL and token

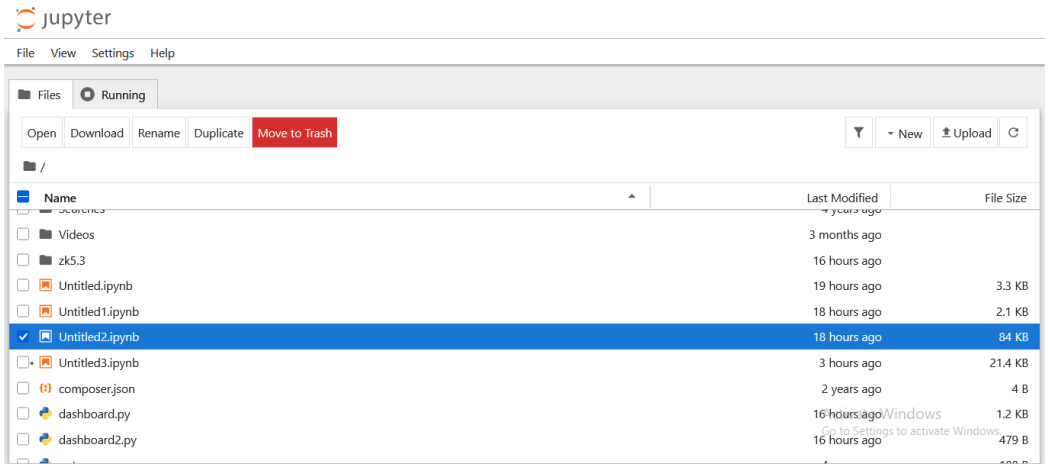

Figure 5 — Jupyter file browser showing the project notebooks

With the environment ready, the notebook defines an on_message callback that fires every time a new MQTT message arrives, decodes the payload, splits it into six sensor values, and appends the row to the CSV dataset using pandas.DataFrame.to_csv in append mode.

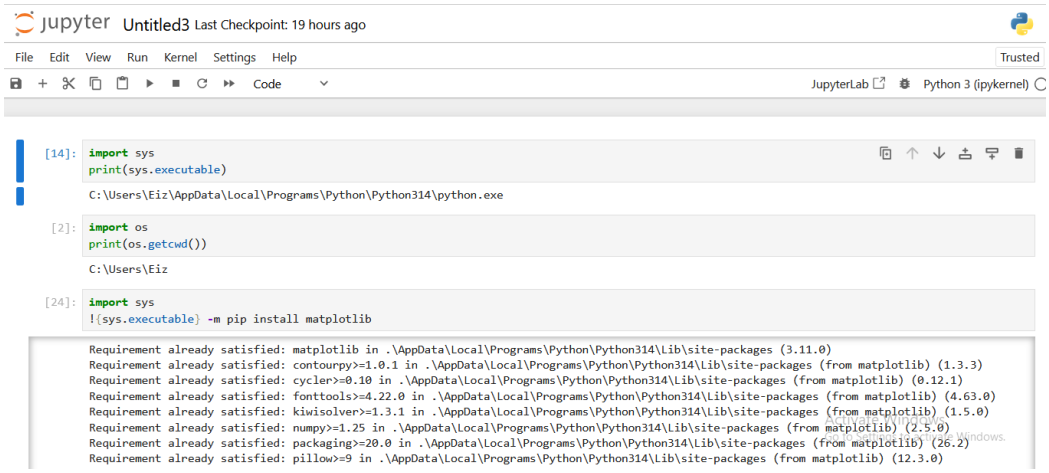

Figure 6 — Verifying the Python environment and installing matplotlib

```
[ ]:
import paho.mqtt.client as mqtt
import pandas as pd
import os
from datetime import datetime

broker = "broker.hivemq.com"
topic = "Registernumber"

file_name = "sensor_data.csv"

# إذا الملف غير موجود، أنشئه مع أسماء الأعمدة
if not os.path.exists(file_name):
    df = pd.DataFrame(columns=[
        "Time",
        "Temperature",
        "Humidity",
        "Gas",
        "Motion",
        "Light",
        "Distance"
    ])
    df.to_csv(file_name, index=False)

def on_message(client, userdata, msg):
    data = msg.payload.decode()
    print("📩", data)

    values = data.split(",")

    if len(values) != 6:
        return

    row = {
        "Time": datetime.now().strftime("%H:%M:%S"),
        "Temperature": float(values[0]),
        "Humidity": float(values[1]),
        "Gas": int(values[2]),
        "Motion": int(values[3]),
        "Light": int(values[4]),
        "Distance": float(values[5])
    }

    df = pd.DataFrame([row])
    df.to_csv(file_name, mode="a", header=False, index=False)

client = mqtt.Client()
client.on_message = on_message

client.connect(broker, 1883, 60)
client.subscribe(topic)

print("📝 Saving data to sensor_data.csv ...")

client.loop_forever()
```

```
C:\Users\Eiz\AppData\Local\Temp\ipykernel_12888\2428788591.py:46: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
📝 Saving data to sensor_data.csv ...
📩 31.90,55.50,843,1,3092,11.68
📩 19.00,55.50,843,0,2800,11.56
```

Figure 8 and Figure 9 — The two previous images contain Python code that receives data from ESP32 sensors via the MQTT protocol and stores it in a CSV dataset

Dataset Generation & Storage

When the Python code is executed and the client.loop_forever() function is called, the notebook begins continuously listening to the MQTT topic, printing all incoming sensor readings while simultaneously saving them in a CSV dataset.

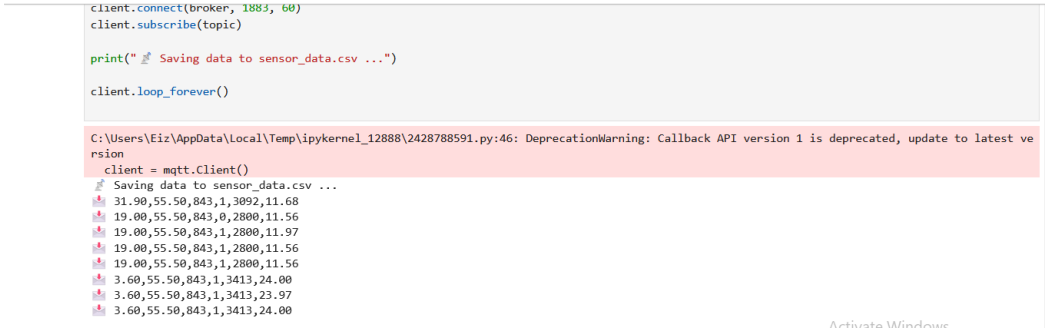

Figure 10 — Live MQTT stream

After operating for a period of time, the CSV file accumulates a series of data from the six sensors. The data in this file was loaded using the pandas.read_csv function to verify its structure and serve as the foundation for the AI-based analysis phase.

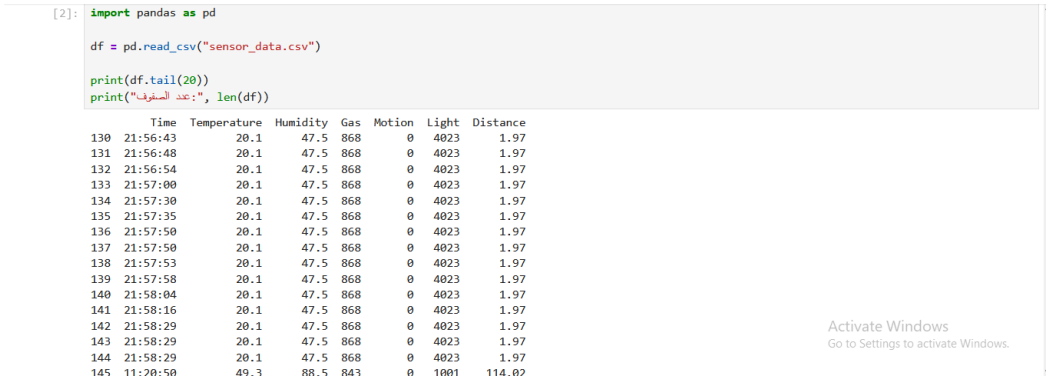

Figure 11 — Loading and inspecting the saved dataset: Time, Temperature, Humidity, Gas, Motion, Light, Distance

Dataset columns

- **Time —** time of the reading
- **Temperature —** degrees Celsius, from the DHT22 sensor
- **Humidity —** relative humidity percentage, from the DHT22 sensor
- **Gas —** raw analog reading from the MQ-2 gas sensor
- **Motion —** binary flag (0/1) from the PIR sensor
- **Light —** raw analog reading from the LDR sensor
- **Distance —** centimeters, from the ultrasonic sensor

> **Next step**
>
> — The previously stored dataset constitutes the exact format required to train an AI model; for instance, a model designed to detect potential risks before they occur by making predictions based on patterns learned from the archived data on such risk scenarios.

04 · PART TWO

Part Two — Real-Time Dashboard Using ThingSpeak

ESP32 → ThingSpeak Cloud → Live IoT Dashboard

In this second, independent part of the project, the ESP32 firmware connects to Wi-Fi, reads all six sensors, and pushes the readings directly to ThingSpeak, a cloud platform for IoT data logging and visualization. No local processing, database, or AI model is involved here — the goal is purely live, human-readable monitoring.

How the data flows

- The ESP32 samples all sensors on a fixed timer interval.
- Readings are packaged and sent over Wi-Fi directly to ThingSpeak.
- ThingSpeak receives the data on its channel API and stores it under six separate fields.
- ThingSpeak automatically renders each field as a live, auto-updating line chart.

Each of the six ThingSpeak fields corresponds to one physical sensor: Temperature, Humidity, Light, Gas, Motion, and Distance. Because ThingSpeak channels are public/shareable, this dashboard can be monitored from any browser or device without installing anything.

> **Result**
> — The ThingSpeak channel receives sensor readings approximately every few seconds, demonstrating a stable end-to-end connection between the ESP32 and the cloud dashboard.

ThingSpeak Channel Results

The screenshots below were captured directly from the live "Full Smart Home with AI" channel on ThingSpeak, showing all six fields updating in real time as the Wokwi simulation ran.

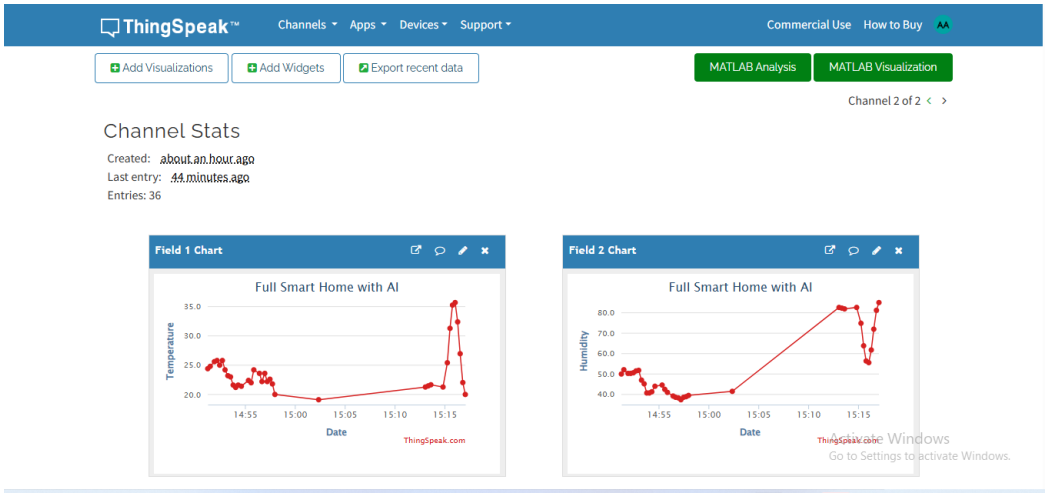

Figure 12 — Field 1: Temperature (°C) and Field 2: Humidity (%)

Figure 13 — Field 3: Gas concentration and Field 4: Motion detection (0/1)

Figure 14 — Field 5: Light intensity and Field 6: Distance (cm)

Reading the charts

- **The motion detection graph** switches cleanly between binary states (0 and 1), confirming reliable PIR sensor operation.
- **All six sensor channels** are synchronized on the same transmission cycle, demonstrating stable real-time cloud communication.

05 · PART THREE

Part Three — Blynk IoT Control Interface

ESP32 → Blynk IoT → User Control & Live Data Display

The third, independent part of the project connects the ESP32 controller to the Blynk IoT platform, which provides a real-time mobile interface for monitoring sensor values and manually controlling actuators. Unlike Parts One and Two, this pipeline is built for direct human interaction rather than data logging or AI processing.

What the interface provides

- Live sensor readings — temperature, humidity, water level, light level, and motion detection are displayed directly on the app.
- Manual control of the window servo motor, fan, lighting, and water pump.
- An AUTO / MANUAL mode switch, letting the user choose between automatic firmware logic and direct manual control.
- A safety status readout ("Save state") that surfaces alarm conditions raised by the firmware.

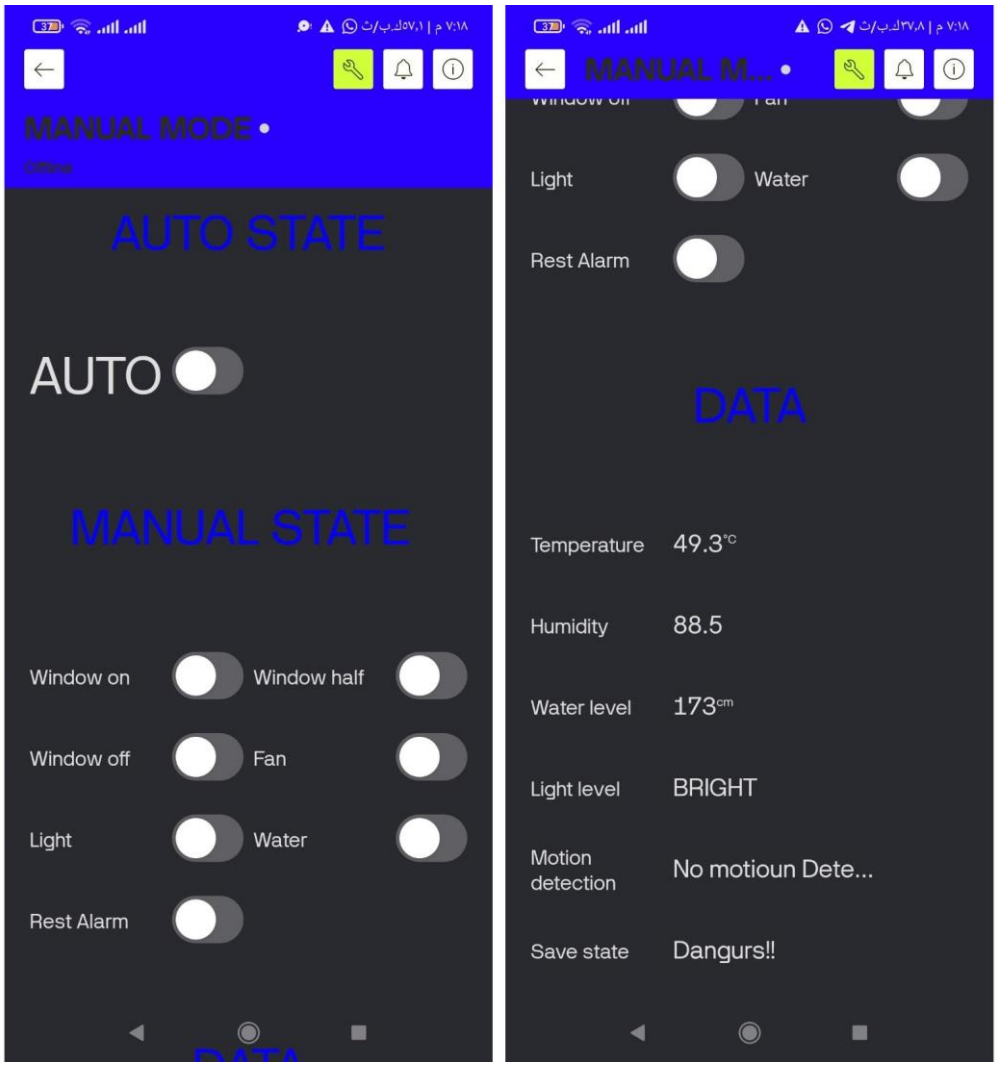

Figure 15 and Figure 16 — The Blynk IoT application interface, showing control buttons on the left and live sensor information on the right

06 · STACK

Software & Hardware Stack

The project combines simulated hardware, cloud IoT services, and a Python-based analytics environment. The table below summarizes every technology used across all three parts of the system.

Technology	Role
ESP32	Main microcontroller
Wokwi	Browser-based circuit simulator for the full sensor network
MQTT	Publish/subscribe protocol carrying data in Part One
Python	Core language for the data pipeline and analysis notebook
Jupyter Notebook	Interactive environment hosting the MQTT client and analysis code
pandas	Structuring, appending, and reading the sensor CSV dataset
paho-mqtt	Python MQTT client library used to subscribe to the broker
ThingSpeak	Cloud channel storing and charting the 6 live sensor fields in Part Two
Blynk IoT	Mobile control and monitoring interface used in Part Three

Layer	Part One	Part Two	Part Three
Transport	MQTT publish/subscribe	HTTP over Wi-Fi	Blynk IoT over Wi-Fi
Destination	Local Jupyter Notebook	ThingSpeak cloud channel	Blynk mobile app
Storage	Local sensor_data.csv	ThingSpeak time-series database	None (live only)
Output	Structured dataset for AI modeling	6 live auto-updating charts	Live readings + manual controls
Best for	Analysis, training, and prediction	Instant, shareable monitoring	Direct user interaction & control

07 · CONCLUSION

Conclusion & Future Work

This project successfully demonstrates three complementary approaches to processing IoT sensor data: a structured, storable data channel via MQTT and Jupyter Notebook for AI (Part One), a real-time public dashboard via ThingSpeak (Part Two), and a mobile control panel via Blynk IoT (Part Three). Together, these approaches cover a wide range of applications, from data-driven intelligence to instant monitoring and direct user control.

Key achievements

- Simulated a complete six-sensor smart-home circuit on Wokwi, including safety actuators (relays, buzzer, alarm LED, servo).
- Built an independent MQTT pipeline delivering sensor data into a Python environment for AI preparation.
- Implemented an automated CSV logging system inside Jupyter Notebook using pandas.
- Established a stable, continuous connection from the ESP32 to a public ThingSpeak channel with six live charts.
- Delivered a Blynk IoT mobile interface for live monitoring and manual control of actuators.
- Prepared a clean, time-stamped dataset ready for AI model training.

Future work

- Train an anomaly-detection model on the gas and motion fields to trigger automatic alerts.
- Add a predictive model that pre-emptively controls the fan and water relays based on temperature and humidity trends.
- Replace the temporary MQTT public broker with a persistent, secured broker for long-term deployment.
- Deploy the trained AI model back onto the ESP32 or an edge device for fully autonomous operation.
- Unify Parts Two and Three into a single mobile dashboard that also surfaces the AI model's predictions from Part One.

"AI Full Smart Home" bridges simulation and real-world IoT engineering — proving that a single sensor network can power three independent, purpose-built pipelines: an AI training channel, a public live dashboard, and a mobile control interface, without any of them interfering with one another.

AI Full Smart Home — IoT & Artificial Intelligence Project Report